Code Supplement: the Spatially-Clustered STEM Model and its Implementation in the `Stem` R Package

A companion to "Latent Spatial Regimes in Spatio-Temporal State-Space Models: Evidence on Local Price Leadership in Italian Fuel Markets"

Paolo Maranzano

November 3, 2026

This document describes how the methods of the manuscript are implemented in the `Stem` package for R (Cameletti et al., 2026; R Core Team, 2026), and how the replication material reproduces its simulation study and its application. It accompanies the package, at https://github.com/PaoloMaranzano/Stem, and is not part of the article: the manuscript, the Computational and Algorithmic Supplement (Supplementary Material A) and the Simulation Supplement (Supplementary Material B) describe the methods and the experiments without reference to the software. Section C1 presents the package, Section C2 says which function and which argument implement each part of the manuscript, Section C3 states how the results are made reproducible, and Section C4 describes the replication material.

C1 The `Stem` package

Everything described in the manuscript is implemented in the `Stem` package for R. The package originates in the implementation of the pooled STEM model accompanying Fassò and Cameletti (2010), and version 2.0.0 adds the spatially-clustered layer, the data-driven penalty scale, the two-step tuning rule, the refit-with-clustering bootstrap and the accelerated estimation algorithms.

A model is set up by `STEM_Model()`, which collects the response matrix, the covariates, the coordinates, the loading matrix and the starting values of the parameters into the object every estimation routine consumes; `STEM_control()` collects the computational settings of every estimation routine — the algorithm, the tolerances and the limits on the iterations — in the spirit of the `control` argument of `optim()`. From there, `STEM_Estimation()` fits the pooled model, `STEM_Signal()` returns its fitted signal and `STEM_Complete()` the conditional means of the missing values, `STEM_Kriging()` produces the spatial predictions at unobserved sites, `STEM_Simulation()` simulates from a fitted model, and `STEM_Bootstrap()` quantifies the uncertainty of the pooled fit.

The spatially-clustered layer follows the same convention and consumes the same `STEM_Model` object. A single entry point, `STEM_Fit()`, takes the data and the hyperparameters and dispatches: with $K = 1$ it calls `STEM_Estimation()` and with $K > 1$ `SCSTEM_Estimation()`, and the ridge on the regression coefficients is switched on by a non-zero `lambda` rather than by a different function. `SCSTEM_Estimation()` fits the model at a given number of regimes and penalty, implementing Algorithm 1 of the manuscript together with the initialization, the minimum-size constraint, the swap pass and the final refit. `SCSTEM_Infocrit()` runs it over a grid of (K, ϕ) and returns the information criteria of every configuration, along with the admissibility diagnostics; `SCSTEM_Select()` applies the two-step rule to that grid. `SCSTEM_Bootstrap()` runs the refit-with-clustering bootstrap,

whose draws, each under its own seed, can be shared among several processes with the same result, and `SCSTEM_BootInference()` post-processes its draws into standard errors, confidence intervals and between-regime tests. `SCSTEM_CV()` runs the optional blocked cross-validation, and `SCSTEM_Signal()` and `SCSTEM_Complete()` are the clustered counterparts of `STEM_Signal()` and `STEM_Complete()`. Each of these returns an object of its own class with a `print()` method, so that a fit can be inspected without extracting components by hand.

The package ships with two example datasets; with four vignettes, covering the pooled model, the spatially-clustered layer, a conceptual map of how the functions connect and the way the filter and the M-step are computed; with the Computational and Algorithmic Supplement as its documentation of the estimation algorithms; and with a `testthat` suite, which checks the log-likelihood, the smoothed states and the maximum against computations on the full covariance of the data. It is available at https://github.com/PaoloMaranzano/Stem.

C2 From the manuscript to the functions

Table C1 lists, for every component of the procedure, the function and the arguments that implement it; the section numbers are those of the manuscript and of the two supplements.

C3 Reproducibility

The package extends the original implementation of the pooled STEM model with the spatially-clustered layer described in the manuscript. Every routine that uses randomness accepts an explicit seed and restores the state of the random number generator on exit, so that the experiments of the manuscript reproduce exactly. The bootstrap gives every draw a seed of its own, derived from the seed of the call, so that its draws do not depend on how many processes share them. The computational settings of every experiment are those of `STEM_control()`, passed explicitly so that no session option can change them.

C4 The replication material

The experiments are reproduced by standalone scripts, separate from the package and needing nothing but R: they are in the folder `papers/sc-stem-local-leader/code` of the repository of the package, beside the dataset of the application and the results of the manuscript, and they install the release 2.0.0 of `Stem` from GitHub, reinstalling it whenever the installed version is another one. Their names say which part of the manuscript they reproduce, and the README of the folder gives every option (Table C2).

The simulation study. The design is set in `Sims_00_Setup.R`: the scenario-variants, the reference levels of the margins and the blocks. Every replication runs in its own R process under a time limit that adapts to its cell, and a replication over the limit is drawn again with the seed of its next attempt. Replication r of every cell is generated from the seed $1000\,r + 1$; a stream s numbers its replications from $1000\,(s-1)+1$, so that streams run on different machines draw different data and their results stack. The study of the manuscript was run with `Stem` at commit `0f7b744`; the release 2.0.0, which the scripts install, reproduces its replications exactly, since what changed after that commit is not used by the study.

```
Rscript Sims_01_Run.R --stream=1 --nrep=100 --cores=7     # one machine
```

Table C1: The components of the procedure and their implementation.

component	function and arguments
the pooled STEM model (Section 2.1)	`STEM_Model()`, `STEM_Estimation()`
the EM algorithm and its accelerations, the stopping rule (Section A1)	`STEM_control(algorithm = "SQUAREM")`, or `"EM"`, `"ECME"`, `"SQUAREM-ECME"`; `em_tol_par`, `em_tol_loglik`, `em_maxit`, `em_stop`; inside the alternation `alt_em_tol_par`, `alt_em_tol_loglik`, `alt_em_maxit`; `nr_maxit`
SC-STEM at fixed (K, ϕ), Algorithm 1 (Sections 2.1–2.4)	`SCSTEM_Estimation(K, phi_penalty)`, or `STEM_Fit()`
the graph of the Potts penalty (Section 2.2)	`knn`; `distance = "geo"` or `"euclidean"`
the sequential label update, or the simultaneous one (Section 2.4)	`label_update = "ICM"` or `"simultaneous"`
the initialization (Section A3)	`init_method = "departures"` (default), `"kmeans"` on the covariate means, `"coordinates"`; `init_partition` for a partition supplied by the user
the scale of the penalty (Section 2.2, Section A3)	`phi_scale = "auto"` (default), `"per-observation"`, `"raw"`
the minimum-size constraint and the swap pass (Section 3.1, Section A4)	`min_cluster_size`, `enforce_min_size`, `swap_pass`
a ridge on the coefficients (not used in the manuscript)	`lambda`, `penalize`, `lambda_scale`, `lambda_by`; `alpha` for the elastic net
the grid and the information criteria (Section 3.2)	`SCSTEM_Infocrit(K_grid, phi_grid)`
the two-step rule (Section 3.2)	`SCSTEM_Select(band = c(0.025, 0.2), criterion = "BIC")`
the predictive validation (not used in the manuscript)	`SCSTEM_CV(scheme)`, with the schemes `"LKLO"`, `"LKTO"`, `"LKLHTO"` and `"random"` of Meyer et al. (2018)
the refit-with-clustering bootstrap (Section 3.3)	`SCSTEM_Bootstrap(B, seed, cores)`, `SCSTEM_BootInference()`
missing observations (Section 3.1, Section A2)	handled by every fit; `STEM_Complete()`, `SCSTEM_Complete()` for the completed response
the fitted signal and the predictions	`STEM_Signal()`, `SCSTEM_Signal()`, `STEM_Kriging()`

```
Rscript Sims_01_Run.R --stream=2 --nrep=100 --cores=7     # another one
Rscript Sims_02_Analysis.R --results=../output/runs/simulations
```

The application. Each run script builds the data of its setting from the weekly prices (the completion of the missing weeks is cached and shared by the three settings) and then fits its models; the cities and fuels of a run can be chosen so that several machines share the work, the seed of a model depending on its city and fuel only. The results of the application were computed with `Stem` at commit `09e9cdb`, which adds to the code of the simulation study SQUAREM for the fits with a ridge and the parallel bootstrap; its code is that of the release 2.0.0.

```
Rscript AppFuels_01_RunSetting1.R --cores=7
Rscript AppFuels_01_RunSetting1.R --cities=RM --fuels=g --cores=7
Rscript AppFuels_04_Analysis.R --results=../output/runs/application
```

Table C2: The replication material.

script	what it does
`Sims_00_Setup.R`	the simulation study of Section 4 of the manuscript and of the Simulation Supplement: the generator, the Monte Carlo of one replication, the runner, and the design (the scenario-variants, the reference levels of the margins, the blocks and the parameter values)
`Sims_01_Run.R`	runs the replications under their time limit, in streams that several machines can run at once
`Sims_02_Analysis.R`	turns the results into the tables and figures of the manuscript and of the Simulation Supplement, and draws those of the design from the definitions of `Sims_00_Setup.R`, so that the description of the design cannot disagree with the design that is run
`AppFuels_00_Setup.R`	the settings and the functions of the application of Section 5 of the manuscript: from the weekly prices to the data of a setting, the model of one city and fuel, the tests and the check of the local maxima
`AppFuels_01_RunSetting1.R`, `AppFuels_02_RunSetting2.R`, `AppFuels_03_RunSetting3.R`	the application, one script per setting: the data of the setting, then, model by model, the grid, the bootstraps and the tests
`AppFuels_04_Analysis.R`	the tables, figures and numbers of Section 5 of the manuscript and of its supplement on the application

References

Michela Cameletti, Francesco Caccia, and Paolo Maranzano. *Stem: Spatio-Temporal Expectation-Maximization Models and their Spatially-Clustered Extension*, 2026. URL https://github.com/PaoloMaranzano/Stem. R package version 2.0.0.

Alessandro Fassò and Michela Cameletti. A unified statistical approach for simulation, modelling, analysis and mapping of environmental data. *Simulation*, 86(3):139–153, 2010. doi: 10.1177/0037549709102150.

Hanna Meyer, Christoph Reudenbach, Tomislav Hengl, Marwan Katurji, and Thomas Nauss. Improving performance of spatio-temporal machine learning models using forward feature selection and target-oriented validation. *Environmental Modelling and Software*, 101:1–9, 2018. doi: 10.1016/j.envsoft.2017.12.001.

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026. URL https://www.R-project.org/.